

4 - dars: Data Extraction va merging, Githubdagi Projectni gitga ulash va yangi commands, Logging va error lardan iborat.

Data Extraction:

Turli xil web-sitelardan etiketga rioya qilingan
to'lda ma'lumotlarni tortib olish protsezi.

• Turli xil tolatdagi dataset/ni birlashti-
rib yagona dataset xosil qilish - merging

• Usullari:

- Bir xil ustun/ (concat)
- Join (common key: join/merge)

Web Scraping (Ma'lumot yig'ish)

web scraping nima?

- Kod yordamida veb-saytlardan ma'lumotlarni
avtomatik ravishda yig'ib olish jarayoni.
- Qo'lda nusxa ko'chirish o'rniga, katta xajm-
dagi ma'lumot/ni tez yig'ib olish jarayonidir.

! Misol: FBref saytidan o'yin natijalari
yoki futbolchilar statistikasini tahlil uchun
yig'ish.

Nega kerak?

- ✓ Takrorlanuvchi ishlarni avtomatlash-tiradi.
- ✓ API orqali mavjud bo'lmagan ma'lumotni o'ladi.
- ✓ Sport, moliya, e-commerce, ilmiy loyi-halarda juda foydali
- ✓ Tartibsiz ma'lumotni CSV/EXCEL for-matiga o'tkazadi.

Asosiy bosqich/:

1. **HTTP** so'rov - sahifaga ulanish (**requests**)
2. **HTML** tahlili - kerakli ma'lumot/ni aj-ratib olish (**BeautifulSoup**, **lxml**)
3. Tartibga solish - jadval shakliga keltirish (**pandas**)
4. Saqlash - CSV, Excel, yoki bazaga yozish.

Kutubxona Lar (Python)

Kutubxona

requests

BeautifulSoup

pandas, read.html

Scrapy

Selenium

Vazifa

HTM yuklab olish

HTM feytlarini o'qish

Jadval larni to'g'ridan-to'g'ri olish

Katta loyiha u-n flask/frame

Javascriptli dinamik sayt/ u-n

Ish tartibi :

1. Maqsadli sayt tamlash
2. Dev tools bilan HTML tuzilishini ko'rish
3. Script yozish (requests + BeautifulSoup)
4. Ma'lumot/ni tozalash
5. Natijani saqlash  (CSV/Baza)

! Biz qidirgan ma'lumotni topganda →
url 'ni copy qilib undan so'ray

[STATIC HTML o'qish]

```
import requests
from bs4 import BeautifulSoup
import pandas as pd
import os
```

Create folder (papka ochamiz)

```
os.makedirs("Data/transfermarkt", exist_ok=True)
```

url = "biz copy qilib olgan sayting urli"

```
headers = {
```

```
    "User-Agent": "Mozilla/5.0"
```

```
}
```

→ Bu Microsoft u-n
 Mac OS u-n alohida
 ko'rish k-k.

fetch page → (yaini birinchi response joratiлади)

```
response = requests.get(url, headers=headers)
soup = BeautifulSoup(response.text, "html.parser")
```

Find table by class

```
table = soup.find("table", {"class": "items"})
```

Bu yer eng muhim joyi bizga harakli ustunda classni masalan saytdan transfer summeleri degan ustundan ma'lumot olmoqli bo'lah o'sha ustunni class nomi nima shuni yozib table olsak bo'ladi.

Headers (ustun nomlarini to'plash)

```
headers = [th.get_text(strip=True) for th in
            table.find_all("th")]
```

Rows (qatorlar)

```
rows = []
```

```
for row in table.find("tbody").find_all("tr"):
    cols = [td.get_text(strip=True) for td in row.
            find_all("td")]
```

```
if cols:
```

```
    rows.append(cols)
```


Create DataFrame

```
df = pd.DataFrame(rows, columns =  
headers[:len(rows[0])])
```

Save CSV

```
file_path = os.path.join("Data/transfermarkt",  
"ronaldo_2016.csv")
```

```
df.to_csv(file_path, index=False,  
encoding="utf-8-sig")
```

Merging

```
df1 = pd.DataFrame({"id": [1, 2], "name": ["A", "B"]})
```

```
df2 = pd.DataFrame({"id": [1, 2], "salary": [100, 200]})
```

```
pd.concat([df1, df2], ignore_index=True)
```

Agar ignore_index=True gilasak

id	name	salary	
0	1	A	bali'g galadi index iku tarfi'g last index n'lsa
1	2	B	
0	3	C	index + ignore_index = True gilasak
1	4	D	

concat → ustun nom/i bir xil bo'lsa ishlatgan ma'qul aks holda NaN qo'yib yuboradi.

Ustun nom/i xer xil bo'lsa merging ishlatish k-k.

```
df1 = pd.DataFrame({"id": [1, 2], "name": ["A", "B"]})
```

```
df2 = pd.DataFrame({"id": [1, 2], "salary": [100, 200]})
```

```
df = pd.merge(df1, df2, on="id")
```

Datasetni o'qiyotib runda o'qishni papiradan chiqarib olinganda ko'pincha xato beradi.

Shunda copy path qilib va (r"copy path") xolatda qilsa k ishlar o'tir.

Buni o'qib ko'rish k-k.

```
df = pd.read_csv(r'copy path.....')
df.info()
```


Hammasini birdaniga binlashtirish u-n

```
import pandas as pd
from pathlib import Path
```

Base folder

```
base = Path(r'copy path of folder')
```

All seasons you want to load

```
seasons = ["1415", "1516", "1617", "1718", "2021"]
```

```
dfs = []
```

```
for s in seasons:
```

```
    file = base / f"metadata-series-a-{s}.csv"
```

```
    if not file.exists():
```

```
        print(f"⚠ File not found: {file}")
```

```
        continue
```

```
    df_temp = pd.read_csv(file)
```

```
    df_temp df
```

```
    df_temp["season"] = s # add season column
```

```
    dfs.append(df_temp)
```

Combine everything

```
df_a = pd.concat(dfs, ignore_index=True)
```

```
print("✓ Combined shape:", df_a.shape)
```

```
print(df_a.head())
```


Ohirida hammasi ni bitta qilib
qo'shish

```
df = pd.concat([df, df_1, df_bundesliga,  
df_epl, df_11], ignore_index=True)
```

```
df.info()
```

Githubdagi Projectni gitga ulash va yangi commands

- Asli nomi Clonning deyimiz
↳ Desktopga folderni auto yaratadi
- Qolganini shu xolatda ketadi.

Bunda bizga eng kerakli narsa shuki biz githubimizdagi
bironta repositoryga kirib uni url olib shuni
clone qilsak kifoya.

git ~~cd~~ clone url → press enter

keyin o'sha project papkasiga kirib

cd project-folder

code . desak bizga VS Code'da ochib
beradi.

Yangi Commands

`mv notebook notebooks` $\Rightarrow$ Bu biz bironfa papkani o'zgartirgichi bolsak shu commands den foydalanamiz. `mv model.py src/` $\Rightarrow$ Bu `model.py` ni `src` papkasiga ko'chiradi

Fayllarda esa:

`git mv app.py apps.py`

Umumiy hulosa nima qilganimiz haqida:

`git log`

Olding ish bilan: hozirgi holatini qanday o'zgarish/ yoki farq/ borligini ko'rsatuvchi:

`git diff`

buyruq	Nima ko'rsatadi?
<code>git log</code>	Commit/ tarixini ko'rsatadi
<code>git diff</code>	Fayllarda qaynan qanday o'zgarish bo'lganini ko'rsatadi

1. `git log` $\Rightarrow$ Bu kim, qachon va nima nom bilan commit qilganini ko'rsatadi.

Masalan: `git log`

commit 8d3f2ab... # commit ID
Author: Murodjon # muallif
Date: Thu Jul 10 # Sana

login page added # Commit message
commit 7f1e234...
Author: Murodjon
Date: Wed Jul 9

Fixed navbar

2. `git diff` $\Rightarrow$ Bu esa kod = ichidagi o'zgarishni ko'rsatadi

Masalan: `git diff`

Before: `name = "Ali"`

After: `name = "Murodjon"`

$\hookrightarrow$ Natija: - `name = "Ali"`
+ `name = "Murodjon"`

- $\rightarrow$ minus bu o'chirilgan qator

+ $\rightarrow$ plus bu qo'shilgan yoki o'zgarigan qator.

Logging va error

- Kodlar yozilganda turli xil xolat/ bôladi:
 - ↳ Warnings
 - ↳ Errors
- Kodningni nazorat qilishni 2 ta asosiy yo'li mavjud
 - ↳ logging
 - ↳ Errors (Exceptions)